

Studio↔Vendor Handshake

Last updated: 2026-05-11

The handshake system governs how studios and vendors establish, maintain, and terminate relationships inside ArtHound. It is the prerequisite gate for all payload dispatch, a studio cannot send an asset payload to a vendor until an active link exists between them.

Concepts

Invite: a studio's request to connect with a specific vendor. Expires after 7 days. Can be resent up to 2 times (each resend resets the 7-day window). Only one pending invite per studio↔vendor pair at a time.

Link: the live relationship created when a vendor accepts an invite. One active link per studio↔vendor pair. Stores the payload format snapshot and review collaboration mode agreed at acceptance time.

Payload format snapshot: a copy of the studio's current payload templates captured at the moment the vendor accepts. Used to detect template drift: if the studio later changes their templates, the snapshot is compared against the vendor's saved mapping to surface new or removed fields.

Ingest template: a vendor's saved default field mapping for payloads from a specific studio (which payload field → which field in the vendor's source tool). Keyed on `vendor_id + studio_id`, not on the link, so it survives cancellation and re-invite.

Cancellation: either party can cancel a link. Cancellation immediately revokes all outstanding (non-ingested) payload dispatches between the pair and writes a full audit trail. Completed (ingested) dispatches are preserved.

Database Schema

`vendors.handle`

Added by this feature. A globally unique, URL-safe handle (e.g. `pixel-forge`) used for vendor discovery. Nullable, existing vendors without a handle won't appear in search. Indexed case-insensitively.

studio_vendor_invites

Column	Type	Notes
id	uuid	PK
studio_id	uuid	FK → studios
vendor_id	uuid	FK → vendors
review_collaboration_mode	text	none isolated collaborative
status	text	pending accepted expired cancelled
resend_count	int	0-2; max resends before studio must cancel and re-invite
expires_at	timestamptz	7 days from creation; extended on each resend
accepted_at	timestamptz	set when vendor accepts
created_by	uuid	FK → auth.users

Unique constraint: partial index on (studio_id, vendor_id) WHERE status = 'pending', one pending invite per pair at a time. Accepted/cancelled rows accumulate as an audit trail.

RLS: studio sees all their own invites (all ops); vendor can SELECT invites addressed to them only (accept/reject go through API, not direct DB write).

studio_vendor_links

Column	Type	Notes
id	uuid	PK
studio_id	uuid	FK → studios
vendor_id	uuid	FK → vendors
invite_id	uuid	FK → the invite that created this link
status	text	active cancelled_by_studio cancelled_by_vendor
payload_format_snapshot	jsonb	Studio's templates at acceptance time
review_collaboration_mode	text	Inherited from invite
cancelled_at	timestamptz	Set on cancellation
cancelled_by	uuid	FK → auth.users

Unique constraint: partial index on (studio_id, vendor_id) WHERE status = 'active', one active link per pair. Cancelled rows accumulate.

RLS: studio sees/manages their own links; vendor sees/manages their own links.

vendor_studio_ingest_templates

Column	Type	Notes
id	uuid	PK
vendor_id	uuid	FK → vendors
studio_id	uuid	FK → studios
link_id	uuid	FK → current link (updated on first save under a new link)
field_mappings	jsonb	{ payload_key: source_field_id, ... }
meta_summary_config	jsonb	Optional: which fields to bundle into a summary block

Unique constraint: (vendor_id, studio_id) , one template per pair, regardless of how many links have existed.

RLS: vendor reads/writes their own templates only. Studios have no access.

link_cancellation_audit

Written when a link is cancelled. Records who cancelled, when, and how many dispatches were revoked.

Column	Notes
link_id	The cancelled link
cancelled_by	User who triggered cancellation
dispatch_count	Denormalized count of revoked dispatches

RLS: both studio and vendor of the cancelled link can SELECT.

link_cancellation_dispatches

Join table: one row per dispatch revoked by a specific cancellation event.

Column	Notes
link_cancellation_id	FK → link_cancellation_audit
dispatch_id	FK → payload_dispatches

Avoids arrays on the audit row; enables indexed lookup per dispatch. Both parties can SELECT.

Full Lifecycle

1. Studio sends an invite

UI: Studio Connections page → "Connect a Vendor" button → `InviteVendorModal`

1. Studio searches by vendor handle (prefix match, min 2 chars, max 10 results): `GET /api/handshake/vendors/search?q=<handle>`
2. Studio selects a vendor and picks a review collaboration mode (`none` / `isolated` / `collaborative`).
3. `POST /api/handshake/invite` , creates the `studio_vendor_invites` row. Blocked if an active link or pending invite already exists for this pair.

2. Vendor receives and reviews the invite

UI: Studio Connections page, pending invites appear at the top.

- `GET /api/handshake/invites/incoming` , vendor fetches their pending invites (non-expired only).
- `GET /api/handshake/invites/{invite_id}/preview` , loads studio name + their current live payload templates so the vendor can see what they'd receive before committing.

3. Vendor accepts (or declines)

Accept flow:

1. `POST /api/handshake/invites/{invite_id}/accept` - Validates invite is still `pending` and not expired. - Fetches the studio's current payload templates and saves them as `payload_format_snapshot` on the new link. - Creates the `studio_vendor_links` row (`status = active`). - Marks the invite `accepted` . - Returns `{ link_id }` .
2. If the studio has payload templates, the vendor is immediately offered a field mapping step: - `GET /api/payloads/link-mapping/{link_id}` , returns payload fields + vendor's source schema + any pre-existing mapping for drift comparison. - Vendor maps each payload field to a field in their source tool. - `PUT /api/handshake/template/{studio_id}` , saves/updates `vendor_studio_ingest_templates` .
3. If the studio has no templates, or the vendor skips mapping, the connection completes without a template (mapping can be set up later).

Decline flow: `POST /api/handshake/invites/{invite_id}/reject` , sets invite `status = cancelled` .

4. Studio manages pending invites

UI: VendorConnections page, pending invites section.

- **Resend:** `POST /api/handshake/invites/{invite_id}/resend` , extends `expires_at` by 7 days, increments `resend_count` . Max 2 resends before studio must cancel and create a new invite.
- **Cancel:** `DELETE /api/handshake/invites/{invite_id}` , sets `status = cancelled` .

5. Active link in use

Once a link is active, studios can dispatch payloads to that vendor via the payload system. The link's `payload_format_snapshot` is used during ingest to detect template drift (new or removed fields since the vendor set up their mapping).

`GET /api/handshake/links`, returns active links for the calling party (studio or vendor), with the counterparty's name resolved.

6. Cancellation

Either party can cancel via `DELETE /api/handshake/links/{link_id}`.

The cancellation sequence (all in one request, no transaction):

1. Find all outstanding dispatches (not revoked, not expired, not yet ingested) between the pair.
2. Bulk-revoke them: set `revoked_at = now()` on all matched `payload_dispatches` rows.
3. Write `link_cancellation_audit` row with the revoked count.
4. Write one `link_cancellation_dispatches` row per revoked dispatch.
5. Set `studio_vendor_links.status` to `cancelled_by_studio` or `cancelled_by_vendor`.

Returns `{ dispatches_revoked: N }` so the UI can show a confirmation message.

Studio cancel UI: shows the count of active dispatches that will be revoked and completed dispatches that are preserved before confirming.

Vendor cancel UI: shows a simpler confirmation with the studio name.

Template Drift Detection

When a vendor views the ingest setup screen for a payload, `compare_payload_snapshots()` in `lib/handshake.py` compares:

- `link_snapshot.field_schema`, the studio's templates at the time the vendor accepted
- `template_mappings`, the vendor's saved `field_mappings` keys

Returns:

```
{
  new_fields:    [...], // in snapshot but not in template - studio added fields
  removed_fields: [...], // in template but not in snapshot - studio removed fields
  unchanged:     [...]  // present in both
}
```

This is surfaced in the ingest mapping UI so vendors know their mapping may be stale.

API Reference

All endpoints are under `/api/handshake`. Requires JWT auth. Studio-only endpoints enforce `require_studio`; vendor-only enforce `require_vendor`; link management accepts either role.

Method	Path	Auth	Description
GET	<code>/vendors/search?q=</code>	studio	Prefix-match vendor handles, max 10 results
POST	<code>/invite</code>	studio	Send invite to a vendor
GET	<code>/invites/sent</code>	studio	List own pending, non-expired invites
POST	<code>/invites/{id}/resend</code>	studio	Extend expiry by 7d (max 2 resends)
DELETE	<code>/invites/{id}</code>	studio	Cancel a pending invite
GET	<code>/invites/incoming</code>	vendor	List pending invites addressed to this vendor
GET	<code>/invites/{id}/preview</code>	vendor	Studio name + their current templates
POST	<code>/invites/{id}/accept</code>	vendor	Accept invite → creates link, returns <code>link_id</code>
POST	<code>/invites/{id}/reject</code>	vendor	Decline invite
GET	<code>/links</code>	studio or vendor	List active links with counterparty resolved
DELETE	<code>/links/{id}</code>	studio or vendor	Cancel link + revoke dispatches + write audit
GET	<code>/template/{studio_id}</code>	vendor	Get saved ingest template for a studio
PUT	<code>/template/{studio_id}</code>	vendor	Upsert ingest template (requires active link)

Frontend

VendorConnections (`frontend/src/pages/VendorConnections.jsx`): Studio side

- Active connections list with vendor name, handle, review mode, connection date.
- Pending invites with expiry countdown (turns warning colour inside 24h), resend/cancel actions.
- Payload Templates section, studios create/edit/delete templates here (controls what fields are dispatched). Templates are managed separately from connections.
- Cancel confirmation modal loads live dispatch counts (outstanding vs. completed) so the studio can see the blast radius before confirming.

StudioConnections (`frontend/src/pages/StudioConnections.jsx`), Vendor side

- Pending invites section, highlighted at top. "Review invite" opens `AcceptInvitePanel`.
- Active connections list with studio name, review mode, connection date, and whether a mapping template has been saved.

- Cancel confirmation modal (simpler, no dispatch count lookup needed on vendor side).

AcceptInvitePanel (inline in `StudioConnections.jsx`)

Three-step flow within a modal:

1. **Preview:** studio name, review mode, list of templates they'll receive. Vendor chooses Accept or Decline.
2. **Mapping:** only shown if the studio has templates. Two-column grid mapping each payload field to a vendor source field. Can be skipped and set up later. If vendor has no source tool connected, shows a prompt to do that first.
3. **Done:** confirmation message.

InviteVendorModal (`frontend/src/components/InviteVendorModal.jsx`): Studio side

Two-step flow:

1. **Search:** debounced handle search (300ms), results selectable.
2. **Confirm:** shows selected vendor, review collaboration mode picker (isolated and collaborative currently disabled/coming-soon). Sends invite.

Design Decisions

No email in the invite flow. Vendors are found by handle lookup within ArtHound. This avoids email deliverability issues and keeps the trust model inside the platform, a studio must know the vendor's handle, which requires some prior relationship.

Snapshot at acceptance, not at dispatch. The `payload_format_snapshot` is taken when the vendor accepts, not when each dispatch is sent. This gives the vendor a stable reference point for their mapping setup, while drift detection flags any subsequent template changes.

Template survives link cancellation. `vendor_studio_ingest_templates` is keyed on `vendor_id + studio_id`, not on the link. If a relationship is cancelled and re-established, the vendor's field mapping is pre-populated from their previous one, they don't have to redo it from scratch.

Cancellation is synchronous and immediate. All dispatch revocations happen in the same request as the link cancellation, not in a background task. This keeps the audit trail atomically complete and means the vendor loses access to outstanding dispatches at the exact moment the studio cancels.

Audit trail is append-only. Cancelled invites and links are never deleted, only status-updated. The `link_cancellation_audit` and `link_cancellation_dispatches` tables provide a full record of what was revoked and when.

Review collaboration mode is vestigial (since 2026-06-12). `review_collaboration_mode` is still set at invite time and copied to the link, but nothing reads it anymore. Cross-org review behaviour is now

configured on the link itself: ad-hoc cross-org reviews are always available on an active link, and the studio's required-submission sequence is the link's review protocol (`review_protocol_def_id` , editable from the Vendors page). See [Asset Reviews](#).